SkillTrack – Student Skill and Progress Tracker

Project Report

Submitted by

Harini Sri P.

Register No: 711524BCS052

Department: CSE-B

1. Abstract

SkillTrack is a lightweight web application designed to help students organize learning tasks and monitor their progress. Users can create tasks, assign each task to a skill category, mark tasks as completed, filter tasks by status, and delete tasks when they are no longer needed. The dashboard automatically calculates the total number of tasks, completed tasks, and overall completion percentage.

The application is built using HTML, CSS, and JavaScript. Browser LocalStorage is used to retain tasks and the selected theme between page refreshes on the same browser. SkillTrack provides a simple, responsive interface that encourages students to plan learning activities and track their progress consistently.

2. Introduction

Students often learn several skills at the same time, such as Python, web development, data analytics, and aptitude. Without a simple tracking method, it can be difficult to remember planned activities and understand how much work has been completed.

SkillTrack addresses this problem with a single dashboard for recording learning tasks and viewing progress. It is a front-end project that can run in a modern web browser without requiring a separate backend server or database.

3. Problem Statement

Students may find it difficult to organize daily learning goals, keep track of pending work, and measure progress across different skills. SkillTrack provides a simple digital interface to record tasks, categorize them, update their completion status, and view progress statistics.

4. Objectives

Create a simple and user-friendly learning dashboard.

Allow users to add tasks and associate them with a skill category.

Allow users to mark tasks as completed and delete tasks.

Provide filters for all, pending, and completed tasks.

Calculate and display task completion statistics automatically.

Save tasks and theme preference in browser LocalStorage.

Provide a responsive layout and a light/dark theme option.

Publish the source code in a GitHub repository.

5. Scope

The current version is intended for individual use in a browser. It supports task management and progress tracking on the same browser profile. It does not include user accounts, cloud synchronization, a server-side database, reminders, or collaboration between multiple users.

6. Technologies Used

HTML5: Defines the page structure, forms, dashboard cards, task list, and semantic content.

CSS3: Controls layout, colors, typography, responsive styling, progress bar appearance, and dark mode variables.

JavaScript: Implements task creation, completion, deletion, filtering, progress calculations, theme switching, and browser storage.

LocalStorage: Stores task data and theme preference locally in the browser.

Visual Studio Code: Used to edit and organize the project files.

Live Server: Can be used during development to serve the HTML page locally.

Git and GitHub: Used for version control and publishing the project source code.

7. System Requirements

Hardware: A computer capable of running a modern web browser and a code editor.

Software: Windows, macOS, or Linux; Visual Studio Code or another text editor; a modern browser such as Chrome, Edge, or Firefox; Git for version control; and an internet connection for accessing GitHub. Live Server is optional and is used for local development.

8. System Design

The application follows a client-side architecture. The HTML page provides the interface, CSS styles the interface, and JavaScript manages user interactions and application state. LocalStorage persists the task list and theme preference in the browser.

Data flow:

1. The user enters a task name and selects a skill.

2. JavaScript validates the task and adds it to the task array.

3. The updated task list is saved to LocalStorage.

4. The task list and dashboard statistics are re-rendered.

5. When the page is reopened or refreshed in the same browser profile, saved tasks are loaded from LocalStorage.

Main modules:

Task input and validation

Task list rendering

Completion and deletion controls

Status filtering

Progress statistics

Theme switching

LocalStorage persistence

9. Implementation

The project is organized into three main files:

index.html: Contains the dashboard header, welcome section, statistics cards, progress bar, task form, task filter, task list container, and footer. It links the stylesheet and JavaScript file.

style.css: Defines the visual design, including dashboard cards, form controls, task rows, progress bar, light and dark theme variables, and a responsive layout for smaller screens.

script.js: Loads saved data from LocalStorage, validates new tasks, creates task objects, updates completion state, removes tasks, filters the displayed tasks, calculates progress, and switches themes. DOM elements are created using JavaScript, and task titles are inserted as text rather than HTML markup.

Each task stores an identifier, title, skill category, and completion status. Overall progress is calculated as the number of completed tasks divided by the total number of tasks, multiplied by 100. If there are no tasks, progress is shown as 0%.

10. How to Run the Project

1. Create a folder named SkillTrack.

2. Create index.html, style.css, and script.js inside the folder.

3. Paste the corresponding project code into each file and save the files.

4. Open the folder in Visual Studio Code.

5. Open index.html with the Live Server extension, or open index.html directly in a modern browser.

6. Add a task, select a skill, and test the completion checkbox, filters, delete button, and theme toggle.

7. Refresh the page to verify that tasks remain saved in the same browser profile.

11. Testing

The following manual tests should be performed before submission:

Test 1 – Add task: Enter a task name and select a skill. Expected result: the task appears in the task list and the total count increases.

Test 2 – Empty task validation: Submit the form without a task name. Expected result: the task is not added.

Test 3 – Complete task: Select a task checkbox. Expected result: the task is marked complete and the completion count and progress percentage update.

Test 4 – Filter tasks: Select Pending or Completed. Expected result: only matching tasks are displayed.

Test 5 – Delete task: Select Delete for a task. Expected result: the task is removed and dashboard statistics update.

Test 6 – Persistence: Add a task and refresh the page. Expected result: the task remains available in the same browser profile.

Test 7 – Theme toggle: Switch between dark and light mode. Expected result: the interface changes theme and the preference is retained where browser storage is available.

Test 8 – Responsive layout: Reduce the browser width. Expected result: the layout adapts for smaller screens.

These are test cases to execute manually; results should be marked as passed only after testing the running application.

12. Advantages

Simple interface suitable for students.

No backend setup is required for the current version.

Immediate visual feedback when tasks are updated.

Progress statistics make task completion easy to understand.

Task data can persist between page refreshes in the same browser.

Responsive design supports desktop and mobile-sized screens.

The project demonstrates core front-end development concepts.

13. Limitations

Data is stored only in the browser and is not synchronized across devices.

Clearing browser site data can remove saved tasks.

There is no login or multi-user support.

There are no notifications, deadlines, or calendar integration.

The current implementation does not use a remote database or backend API.

The external font may require an internet connection; fallback fonts are available.

14. Future Enhancements

Add user registration and login.

Integrate a backend API and database for cloud storage.

Add due dates, reminders, and priority levels.

Show progress by individual skill category.

Add weekly and monthly progress charts.

Export tasks and progress as CSV or PDF.

Add search and sorting options.

Deploy the website so it can be accessed through a public URL.

15. Conclusion

SkillTrack demonstrates how HTML, CSS, and JavaScript can be combined to build a useful student productivity application. It allows users to organize learning tasks, update completion status, filter tasks, and monitor overall progress through a simple dashboard. LocalStorage provides basic persistence without requiring a backend. The project can be extended in the future with cloud storage, authentication, reminders, and detailed analytics.